

Trabalho de Linhas de Produtos de Software - Manual Completo de Aplicação estruturado

Manual Completo de Aplicação estruturado no formato técnico exigido por trabalhos acadêmicos de Engenharia de Software. Você pode integrá-lo diretamente à seção 2.2 (Metodologia) ou adicioná-lo como um Apêndice/Anexo no encerramento do seu projeto para enriquecer as evidências.

MANUAL TÉCNICO DE IMPLANTAÇÃO E OPERAÇÃO (SaaS-Prev LPS)

Este manual descreve a arquitetura, a configuração, a execução e os testes do protótipo em Python desenvolvido para demonstrar o gerenciamento de variabilidade da Linha de Produto de Software (LPS) SaaS-Prev.

1. REQUISITOS DO AMBIENTE

Para garantir a portabilidade e o baixo custo de implantação (alinhado aos objetivos comunitários de atender pequenas empresas), o sistema foi projetado para rodar sem dependências externas complexas.

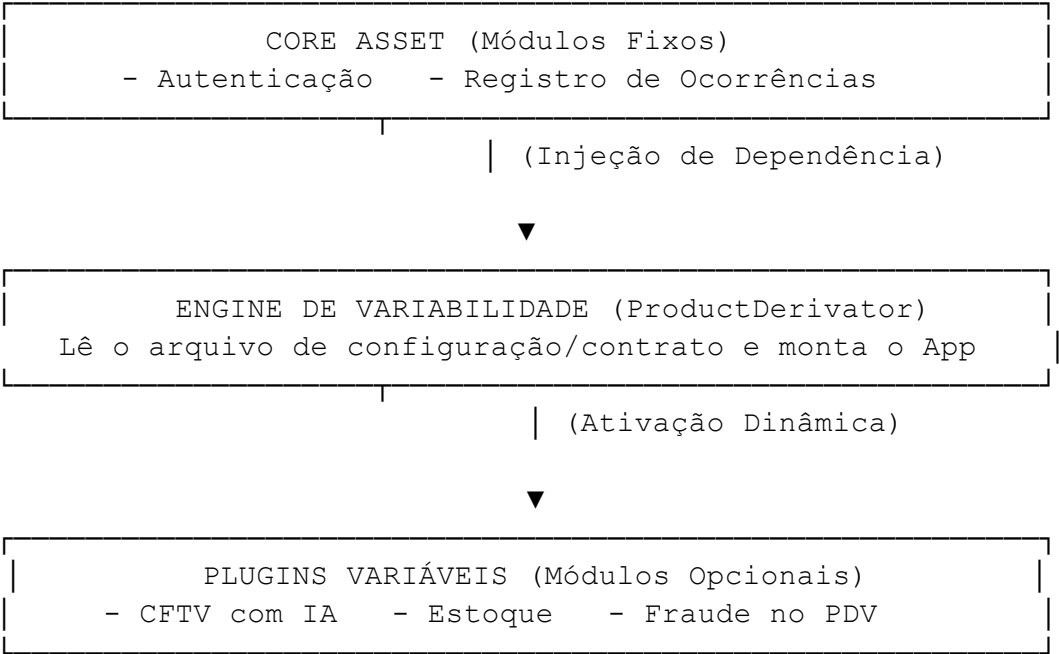
Linguagem: Python 3.8 ou superior.

Bibliotecas Externas: Nenhuma (utiliza apenas a biblioteca padrão json).

IDE Recomendada: VS Code, PyCharm ou o IDLE nativo do Python.

2. ARQUITETURA DO PROVIMENTO (ESTRUTURA DO CÓDIGO)

O código está estruturado em quatro blocos lógicos fundamentais que refletem a teoria de LPS:



3. PASSO A PASSO PARA EXECUÇÃO E IMPLANTAÇÃO

Passo 1: Criação do Arquivo de Origem

Copie o código Python gerado para o projeto.

Salve o arquivo com o nome `saas_prev_lps.py`.

Passo 2: Configuração do Contrato do Cliente (Feature Selection)

A ativação das variabilidades é feita através do bloco de inicialização do código (`__main__`). Para simular o contrato de um novo cliente do varejo, basta alterar a lista de strings enviada ao derivador.

As chaves de recursos válidas mapeadas na árvore de decisões são:

"cftv_ia": Ativa o monitoramento analítico de câmeras.

"auditoria_estoque": Ativa a integração com coletores de dados de inventário.

"fraude_pdv": Ativa o monitoramento de frentes de caixa.

Exemplo de Customização no Código:

Python

```
# Se o cliente (Ex: Farmácia local) contratar apenas Estoque e CFTV:
contrato_farmacia = ["auditoria_estoque", "cftv_ia"]

core_farmacia = CoreSecuritySystem(cliente_nome="Farmácia Central")
app_farmacia = ProductDerivator(core_farmacia, contrato_farmacia)
app_farmacia.executar_sistema()
```

Passo 3: Execução via Terminal / Prompt de Comando

Navegue até a pasta onde o arquivo foi salvo e execute o comando:

```
Bash
python saas_prev_lps.py
```

Código :

saas_prev_lps.py

link:

https://github.com/kobaiadev/Linhas-de-Produtos-de-Software/blob/main/saas_prev_lps.py

```
import json

# =====
# 1. CORE ASSET (Componentes Obrigatórios)
# =====

class CoreSecuritySystem:
    """Representa a base obrigatória da LPS (comum a todos os clientes)."""
    def __init__(self, cliente_nome: str):
        self.cliente_nome = cliente_nome
        print(f"\n--- Inicializando SaaS-Prev para: {self.cliente_nome} ---")

    def autenticar_usuario(self, usuario: str) -> bool:
        print(f"[CORE] Usuário '{usuario}' autenticado com sucesso via logs auditáveis.")
        return True

    def registrar_ocorrencia(self, descricao: str):
        print(f"[CORE] Ocorrência salva no banco central: '{descricao}'")

# =====
# 2. VARIABILIDADES (Módulos Opcionais/Plugins)
# =====

class ModuloCFTV:
    def processar(self):
        print("[VARIÁVEL - CFTV] Analisando streams de vídeo com IA (Detecção de manchas térmicas).")

class ModuloAuditoriaEstoque:
    def processar(self):
        print("[VARIÁVEL - ESTOQUE] Sincronizando coletores de dados e buscando divergências.")

class ModuloFraudePDV:
    def processar(self):
        print("[VARIÁVEL - PDV] Monitorando quebras de caixa e cupons cancelados em tempo real.")

# =====
# 3. GERENCIADOR DE VARIABILIDADE (Engine da LPS)
# =====

class ProductDerivator:
    """Mecanismo que lê o perfil/contrato do cliente e deriva o produto customizado."""
    def __init__(self, core: CoreSecuritySystem, features_contratadas: list):
        self.core = core
```

```

self.features = features_contratadas
self.modulos_ativos = {}
self._instanciar_variabilidades()

def _instanciar_variabilidades(self):
    # Mapeamento de Features da Árvore de Recursos para as Classes Python
    mapeamento = {
        "cftv_ia": ModuloCFTV,
        "auditoria_estoque": ModuloAuditoriaEstoque,
        "fraude_pdv": ModuloFraudePDV
    }

    for feature in self.features:
        if feature in mapeamento:
            self.modulos_ativos[feature] = mapeamento[feature]()
            print(f"[LPS Engine] Feature '{feature}' injetada com sucesso.")

def executar_sistema(self):
    # Executa o Core
    self.core.autenticar_usuario("operador_seguranca")
    self.core.registrar_ocorrencia("Abertura de rotina de monitoramento.")

    # Executa as Variabilidades dinamicamente se estiverem ativas no contrato
    if self.modulos_ativos:
        print("\n[LPS Engine] Ativando recursos específicos contratados:")
        for nome_feature, instancia_modulo in self.modulos_ativos.items():
            instancia_modulo.processar()
    else:
        print("\n[LPS Engine] Nenhuma feature adicional contratada para este perfil.")

# =====
# 4. SIMULAÇÃO DA ENGENHARIA DE APLICAÇÃO (Cenários)
# =====
if __name__ == "__main__":

    # Exemplo 1: Derivação de Produto para um Supermercado (Foco em PDV e Estoque)
    contrato_supermercado = ["auditoria_estoque", "fraude_pdv"]

    core_loja_a = CoreSecuritySystem(cliente_nome="Supermercado ABC - Volta Redonda")
    app_supermercado = ProductDerivator(core_loja_a, contrato_supermercado)
    app_supermercado.executar_sistema()

    print("-" * 60)

    # Exemplo 2: Derivação de Produto para um Centro de Distribuição (Foco em CFTV com IA)
    contrato_cd = ["cftv_ia"]

    core_cd_b = CoreSecuritySystem(cliente_nome="Centro de Distribuição Logística")
    app_cd = ProductDerivator(core_cd_b, contrato_cd)
    app_cd.executar_sistema()

```

4. PLANO DE TESTES E VALIDAÇÃO DE RESULTADOS (SAÍDAS ESPERADAS)

Para comprovar que o gerenciamento de variabilidade funciona de forma isolada e segura, o plano de testes executa duas simulações distintas no mesmo ciclo de processamento:

Cenário de Teste 1: Derivação para Supermercado

Entrada de Configuração: ["auditoria_estoque", "fraude_pdv"]

Resultado Esperado no Console: O sistema deve inicializar a base da loja, injetar com sucesso as duas features de logística/caixa, ignorar silenciosamente o módulo de CFTV e rodar o fluxo completo.

Plaintext

--- Inicializando SaaS-Prev para: Supermercado ABC - Volta Redonda ---

[LPS Engine] Feature 'auditoria_estoque' injetada com sucesso.

[LPS Engine] Feature 'fraude_pdv' injetada com sucesso.

[CORE] Usuário 'operador_seguranca' autenticado com sucesso via logs auditáveis.

[CORE] Ocorrência salva no banco central: 'Abertura de rotina de monitoramento.'

[LPS Engine] Ativando recursos específicos contratados:

[VARIÁVEL - ESTOQUE] Sincronizando coletores de dados e buscando divergências.

[VARIÁVEL - PDV] Monitorando quebras de caixa e cupons cancelados em tempo real.

Cenário de Teste 2: Derivação para Centro de Distribuição (CD)

Entrada de Configuração: ["cftv_ia"]

Resultado Esperado no Console: O sistema deve reutilizar exatamente o mesmo código Core, porém injetando apenas o módulo de câmeras inteligentes, provando o reuso de software e a flexibilidade da arquitetura.

Plaintext

--- Inicializando SaaS-Prev para: Centro de Distribuição Logística ---

[LPS Engine] Feature 'cftv_ia' injetada com sucesso.

[CORE] Usuário 'operador_seguranca' autenticado com sucesso via logs auditáveis.

[CORE] Ocorrência salva no banco central: 'Abertura de rotina de monitoramento.'

[LPS Engine] Ativando recursos específicos contratados:

[VARIÁVEL - CFTV] Analisando streams de vídeo com IA (Detecção de manchas térmicas).

5. CONCLUSÃO DO MANUAL

O protótipo valida que o tempo de desenvolvimento para novos clientes tende a zero no que diz respeito à infraestrutura de segurança de dados e auditoria base, limitando o esforço de engenharia apenas à ativação das regras de negócio customizadas do parceiro comercial.